claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_b0e273c7-38a\agent-aeaf3aab791d81d53.jsonl`
- SHA-256: `873c9871459823180296eb61ea577a73902a45a3652617585eff0f1974acc84e`
- Source modified: `2026-06-21T07:59:53+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b0e273c7-38a`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T07:56:51.764Z)

```
You are reviewing milestone M2 of COMPUTE_DECOUPLED_SERVING in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
Spec: docs/COMPUTE_DECOUPLED_SERVING/README.md §7 row M2 = "input_backend/input_root; wire main
CLI/API input through StorageRef.parse_uri (local=identity); storage-fake tests". M2 must be
SAFE and
default-OFF: with the default config (storage.input_backend='local', input_root=''), the loaded
input
path must be byte-identical to before ? a bare/local path is read in place with NO copy. A
bucket/Drive
URI (gs://, s3://, gdrive://) is fetched to a local scratch file before processing.

The change (read all of these + git diff against origin/master):
- backend/config/models.py: StorageConfig gains input_backend + input_root.
- backend/storage/ref.py: resolve_input_uri() (precedence: explicit scheme > anchored local path
>
  input_root join > input_backend prefix > bare-local).
- backend/storage/__init__.py: acquire_local_input() (local=identity, remote=mkdtemp+fetch) +
  input_is_local().
- backend/main.py: _execute_processing() (API/job path) + the CLI --video-path block both
  acquire-then-use the local path (keeping the original URI for the DB record + report);
  /api/process skips the local os.path.exists() gate for a non-local URI.
- tests: tests/test_input_source.py (new) + 3 tests in tests/test_api_endpoints.py.

Reference the worker's already-shipped pattern (backend/worker/__main__.py:91-105 cmd_infer) and
the
path-safety helpers (backend/utils/paths.py: validate_input_path / allowed_video_root). Ground
EVERY
claim in code with file:line. Give the exact triggering input + wrong-vs-right behaviour. Prefer
FEWER,
HIGH-CONFIDENCE findings. A clean bill is a valid, good result.

LENS: test quality & coverage. For each NEW behaviour, is there a test that
FAILS if it regressed? Specifically: is the CLI block (main.py, not just the API) covered? Is
hosted-mode
(allowed_video_root set) rejection of a non-local URI tested? Is the identity-no-copy guarantee
asserted
at the integration level (segmenter receives the exact local path)? Are there OS-dependent
assertions that
would break on the cross-OS CI (ubuntu/macos) ? e.g. os.path.isabs, backslash handling, tempdir
paths?
Name any missing high-value test with its exact assertion.
```

2. assistant (2026-06-21T07:56:54.809Z)

I'll review M2 of COMPUTE_DECOUPLED_SERVING with a focus on test quality & coverage. Let me start by reading the spec and the relevant code.

3. user (2026-06-21T07:56:58.660Z)

```
---BRANCH---
feat/serving-m1-deployment-profile
---DIFF STAT vs origin/master---
 backend/config/models.py    | 10 ++++++++
 backend/main.py             | 33 +++++++++++++++++++++-------
 backend/storage/__init__.py | 45 ++++++++++++++++++++++++++++++++++++++--
 backend/storage/ref.py      | 30 ++++++++++++++++++++++++++
 tests/test_api_endpoints.py | 58 +++++++++++++++++++++++++++++++++++++++++++++++++++++
 5 files changed, 165 insertions(+), 11 deletions(-)
```

4. user (2026-06-21T07:56:59.175Z)

```
1       # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3       > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
5       > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
6       > **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7       > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an afterthought.
8       > **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10      ---
11
12      > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).
13
14      ## 0. TL;DR
15      The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17      ## 1. Problem & goal
18      - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
19      - **The two user-facing outcomes** (owner, 2026-06-21):
20        - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
```

```
21        - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
GPU**, run inference **and** stitching **completely locally**.
22      - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup
choice**, never a code branch in the core.
23
24      ## 2. Decisions locked
25
26      | # | Decision | Source | Implication |
27      |---|----------|--------|-------------|
28      | D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile
branch inside it**. | design `[verified]` | All profile difference lives in config + the two
registries. |
29      | D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` +
`compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014
| Realizes PLATFORM_ARCHITECTURE's `local\|hosted\|byo_worker`. |
30      | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI);
`byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31      | D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so
CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would
hide real cost. |
32      | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory);
**detect** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the
rally/window level, never CSV bytes. |
33      | D6 | **Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller
change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-
convention]] | Zero-regression discipline. |
34      | D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an
async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that
~$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane
only on a UX complaint. |
35      | D8 | **Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP
billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore
market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |
36      | D9 | **Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud
Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to
the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity
system. |
37      | D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training
rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is
the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ?
reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on
**DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads
**gated to M9** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties
[[paper-coauthor-aim]] / PERSONALIZATION flywheel. |
38      | D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model
clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the
owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a
training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39      | D12 | **Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented,
adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video
workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones
to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11
floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) +
`data_pipeline/` + the eval/experiment harness. |
40      | D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs
**local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure
core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ?
your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the
**rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile,
switching is *between* jobs. |
41      | D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a
**Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel
lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api`
(OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount**
```

backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
| D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |

3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*

This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:

ADR / doc	Contributed	Subtlety carried into this project
ADR-005	Permissive licensing (MIT/Apache-2.0/BSD only)	The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean.
ADR-008	Owned-model topology; **train==serve featurizer single-sourcing**	The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**.
ADR-009	KhelSutra Guru; **local-first delivery**	The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class.
ADR-010	DECOUPLED: DO?GCP pivot	The cloud-compute origin (GPU + co-located bucket).
ADR-011	DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture	**This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**.
ADR-012	GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever	The serving substrate + the 7-step seed scope this expands.
ADR-013	Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only	The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target.
? ADR-014 (new)	**Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered	This doc. **Refines** ADR-011/012, does **not** supersede them.
PLATFORM_ARCHITECTURE.md	The `ComputeBackend`/`StorageBackend` registries	The abstraction this realizes in code.
VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md	L0 fully-local tier / `LocalDesktop` backend	The truly-local profile = these, productionized.
07-COMPUTE-ABSTRACTION (archived)	`DeviceContext`/`DeviceProfile` (designed)	WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes.
DEPLOY_REPORT_GCP_2026-06-20 (archived)	First real L4 run; **cross-GPU sub-pixel parity finding**	Why D5 (parity at window level, not CSV bytes).

4. Design / architecture

See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**

4.1 The core contract (must stay byte/behaviour-identical) `[verified]`

1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.

Non-goal (the trap to avoid): *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).

71
72   ### 4.2 The deployment profiles
73 | Profile | Compute | Storage | Orchestration | When |
74 |---|---|---|---|---|
75 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
76 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
77 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
78 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
79
80   ### 4.3 Profile selection (the "startup/setup config") `[design]`
81   ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
82   - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
83   - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
84   - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
85   - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
86   - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
87
88   ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
89   DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)
90
91   ### 4.5 Reuse map ? most of the seams already exist `[verified]`
92   **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.
93
94   ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
95   Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip

Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:
96   1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
97   2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
98   3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
99   4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
100   This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.
101
102   ## 6. Honest limits ? what this does NOT do / prove
103   - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
104   - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
105   - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
106   - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.
107
108   ## 7. Deliverables & progress tracker ? **source of truth**
109   Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.
110
111 | ID | Deliverable | Depends | Gated? | Status | PR |
112 |----|-------------|---------|--------|--------|----|
113 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **? APPROVED 2026-06-21** |
114 | M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
115 | M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? | ? |
116 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? | ? |
117 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | ? |
118 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |
119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
121 | M8 | Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
124 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the

golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
125 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
126
127   **Testing strategy is a first-class deliverable** ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).
128
129   ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
130   **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
131   1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
132   3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
133   4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
134   5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
135   6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).
136
137   **Still open (non-gating, smaller):**
138   2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics say otherwise.*
139   7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
140   8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?
141   9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and route larger files elsewhere?
142   10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep `CLOUD_RUN_GPU_SERVING/` ? confirm.
143
144   ## 9. Definition of done
145   - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
146   - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
147   - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
148   - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.
149   - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.
150   - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
151
152   ## 10. References
153   **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014** (`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`, `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`, `LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:** `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`, `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`, `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`, `backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-mappers ? synthesis; 58 findings).
154
155   ---
156   ### Changelog

```
157     - 2026-06-21 ? created from the design workflow output; M0 in progress.
158     - 2026-06-21 ? **M0 APPROVED**; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
159
```

5. assistant (2026-06-21T07:57:01.145Z)

Let me read the core implementation files and the new tests.

6. user (2026-06-21T07:57:02.765Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 3612bd8..a31c9da 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -302,6 +302,16 @@ class StorageConfig(BaseModel):
     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
     # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
     output_root: str = ""
+    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
+    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read IN
PLACE
+    # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
+    # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` +
"x.mp4"
+    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
+    # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the input
+    # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
+    # scratch dir before processing (see ``backend.storage.acquire_local_input``).
+    input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    input_root: str = ""
     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
diff --git a/backend/main.py b/backend/main.py
index 942217e..33c7995 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -32,6 +32,7 @@ from backend.utils.paths import resolve_under_root, safe_output_filename,
valida
 from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  # new
typed config
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
+from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)

 app = FastAPI(title="Sports Highlight Indexer API")

@@ -151,13 +152,18 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
     notify = progress or (lambda stage: None)

     notify("hashing")
-    video_id = compute_video_id(req.video_path)
+    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity
?
```

```
+    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
+    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
+    # consumers (hash / validators / segmenter) use the resolved local path.
+    local_video = storage.acquire_local_input(req.video_path, getattr(global_config, "storage",
None))
+    video_id = compute_video_id(local_video)
     was_reingest = db_instance.get_video(video_id) is not None

     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
     notify("validating")
     logger.info(f"Running validations for {req.video_path}...")
-    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
+    val_results, val_context = registry.run_all_with_context(local_video, global_config)
     failures = [r for r in val_results if not r.passed]

     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
@@ -205,7 +211,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
         segmenter = segmenter_cls(db_instance, global_config)
         try:
-            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
+            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
         except Exception:
             # A raising segmenter must not leave half-written rows: restore the pre-run
             # state (same destroy-after-confirm contract as the Failure branch), then let
@@ -265,8 +271,13 @@ def process_video(req: ProcessRequest):
         validate_input_path(req.video_path, allowed_root=allowed_root)
     except ValueError as e:
         raise HTTPException(status_code=400, detail=str(e)) from e
-    if not os.path.exists(req.video_path):
-        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
+    # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive URI can't
be
+    # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it (a
fetch miss
+    # fails the job loudly). validate_input_path above still runs for every input: its null-
byte guard
+    # always applies, and a configured allowed_video_root (hosted mode) rejects a non-local
URI.
+    if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
+        if not os.path.exists(req.video_path):
+            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
     if req.segmenter and not segmenter_registry.get(req.segmenter):
         available = list(segmenter_registry.list_available().keys())
         raise HTTPException(
@@ -577,16 +588,20 @@ def main():
     # CLI processing mode (no web UI needed)
     if args.video_path and not args.server:
         print(f"[CLI] Processing video file: {args.video_path}")
-        if not os.path.exists(args.video_path):
+        # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
+        # The local-existence fast-fail only applies to a local input; a remote fetch fails
loudly.
+        storage_cfg = getattr(global_config, "storage", None)
+        if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
             print(f"[FAIL] Video file not found: {args.video_path}")
             return
+        local_video = storage.acquire_local_input(args.video_path, storage_cfg)
```

```
-        video_id = compute_video_id(args.video_path)
+        video_id = compute_video_id(local_video)
         was_reingest = db_instance.get_video(video_id) is not None

         # Validate (with-context variant surfaces ffprobe_meta for the report)
         print("[CLI] Validating...")
-        val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
+        val_results, val_context = registry.run_all_with_context(local_video, global_config)
         failures = [r for r in val_results if not r.passed]

         # Save video status
@@ -619,7 +634,7 @@ def main():
             segmenter_actual = seg_name
             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
             segmenter = segmenter_cls(db_instance, global_config)
-            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
+            result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
             segmentation_ran = True

             if isinstance(result, Failure):
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 78b4795..656274f 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -13,17 +13,58 @@ passthrough. See ``docs/DECOUPLED_COMPUTE/``.
 from __future__ import annotations

 import os
+import tempfile
 from typing import Any, Dict, Iterator, Optional

 from backend.storage.base import StorageBackend
-from backend.storage.ref import StorageRef, StorageStat, parse_uri
+from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

 __all__ = [
-    "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
+    "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
     "get_storage", "fetch", "put", "list_uris",
+    "acquire_local_input", "input_is_local",
 ]


+def _storage_dict(storage_config: Any) -> Dict[str, Any]:
+    """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
+    if storage_config is None:
+        return {}
+    if hasattr(storage_config, "model_dump"):
+        return storage_config.model_dump()  # type: ignore[no-any-return]
+    return dict(storage_config)
+
+
+def input_is_local(raw: str, storage_config: Any = None) -> bool:
+    """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ? i.e. a
+    plain on-disk file the caller can stat/validate directly. False for a bucket / Drive URI
+    that must be fetched first. Lets callers keep their local-only checks (os.path.exists,
+    allowed_video_root) for the local case and skip them for a remote source."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                            input_root=sc.get("input_root", ""))
+    return parse_uri(uri).backend == "local"
+
+
```

```
+def acquire_local_input(raw: str, storage_config: Any = None, *,
+                        scratch_parent: "str | None" = None) -> str:
+    """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for processing.
+
+    Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
+    byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a bare name
under a
+    non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the
local
+    path returned. The scratch copy is left in place (a pure intermediate, regenerable from the
+    source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims it."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                            input_root=sc.get("input_root", ""))
+    ref = parse_uri(uri)
+    if ref.backend == "local":
+        return ref.key  # identity ? no temp, no copy, byte-identical to passing the path
directly
+    scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
+    return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
+
+
 def _backend_class(name: str):
     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
     if name == "local":
diff --git a/backend/storage/ref.py b/backend/storage/ref.py
index ad5d784..60e2430 100644
--- a/backend/storage/ref.py
+++ b/backend/storage/ref.py
@@ -66,3 +66,33 @@ def parse_uri(uri: str) -> StorageRef:
         bucket, _, key = rest.partition("/")
         return StorageRef(scheme, bucket, key, s)
     raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
+
+
+def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "") -> str:
+    """Resolve a user-supplied input reference into a storage URI (COMPUTE_DECOUPLED_SERVING
M2).
+
+    Precedence (first match wins):
+      1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
+      2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always local).
+      3. A bare/relative name + a non-empty ``input_root`` ? joined under it
+         (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
+      4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
+         ``"gcs://bucket/x"``).
+      5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
+
+    So the default (``input_backend="local"``, ``input_root=""``) is an identity passthrough
for every
+    path used today, preserving byte-for-byte behaviour.
+    """
+    s = str(raw)
+    if _SCHEME_RE.match(s):           # 1. explicit scheme wins
+        return s
+    # 2. an anchored LOCAL path is always local, never joined. Detect this OS-INDEPENDENTLY ?
+    #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a POSIX-
style
+    #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows drive
prefix "X:".
+    if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
+        return s
```

```
+    if input_root:                    # 3. join a relative name under the configured root
+        return input_root.rstrip("/") + "/" + s.lstrip("/")
+    if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the declared
backend
+        return f"{input_backend}://{s}"
+    return s                          # 5. bare/relative local path ? unchanged
```

7. user (2026-06-21T07:57:04.307Z)

```
diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index 001c9fc..ebedddf 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -334,3 +334,61 @@ def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
     jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
     assert jobf["result"]["status"] == "failed"
     # empty-run similar (would preserve prior via reingest safety in both paths)
+
+
+# ---------------------------------------------------------------- M2: URI-aware input
+class _CaptureSeg:
+    """Records the video_path the segmenter actually received (M2: must be a LOCAL path)."""
+    last_path = None
+
+    def __init__(self, db, cfg):
+        self.db = db
+
+    def process_video(self, video_id, video_path, *a, **k):
+        _CaptureSeg.last_path = video_path
+        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
+        return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}]
+
+
+def test_process_local_input_passes_identity_path(env):
+    """Default (local) input must reach the segmenter as the EXACT same path ? identity, no
copy
+    (the M2 zero-behaviour-change guarantee for today's local workflow)."""
+    client, db, tmp_path, mp = env
+    vid = _video(tmp_path)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
+    assert r.status_code == 202
+    assert _CaptureSeg.last_path == str(vid)
+
+
+def test_process_local_missing_file_still_404(env):
+    """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is preserved
+    for local paths (only non-local URIs skip it)."""
+    client = env[0]
+    r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
+    assert r.status_code == 404
+
+
+def test_process_nonlocal_uri_skips_exists_and_fetches(env):
+    """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the worker
+    fetches it to a local scratch file and the segmenter sees that LOCAL path, not the URI."""
+    client, db, tmp_path, mp = env
+
+    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+        with open(dst, "wb") as fh:
+            fh.write(b"fetched-video-bytes")
+        return dst
```

```
+
+    mp.setattr(m.storage, "fetch", fake_fetch)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
+    assert r.status_code == 202                              # NOT 404 ? non-local skips the
exists gate
+    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
+    assert job["status"] == "done"
+    assert _CaptureSeg.last_path is not None
+    assert not _CaptureSeg.last_path.startswith("gs://")     # a fetched LOCAL path, not the
URI
+    assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
===NEW TEST FILE===
-rw-r--r-- 1 avidu 197609 6055 Jun 21 13:23 tests/test_input_source.py
```

8. assistant (2026-06-21T07:57:07.100Z)

```
The new test file exists but is NOT in the diff against origin/master ? meaning it's untracked.
Let me check git status and read the new test file.
```

9. user (2026-06-21T07:57:08.921Z)

```
 M backend/config/models.py
 M backend/main.py
 M backend/storage/__init__.py
 M backend/storage/ref.py
 M tests/test_api_endpoints.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_input_source.py
===is test_input_source.py tracked?===
===show in diff if staged===
```

10. user (2026-06-21T07:57:09.199Z)

```
1       """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
2       acquire_local_input.
3
4       The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5       identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6       routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7       monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9       OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10      '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11      (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12      """
13      import os
14
15      import pytest
16
17      from backend import storage
18      from backend.storage import acquire_local_input, input_is_local
19      from backend.storage.ref import resolve_input_uri
20
```

```
21
22      # ------------------------------------------------------------- resolve_input_uri
23      @pytest.mark.parametrize("raw,ib,ir,expected", [
24          # 1. explicit scheme always wins (even over a non-local backend + root)
25          ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
26          ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
27          ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
28          ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
29          # 2. an anchored local path (leading slash, or drive prefix) is always local, never
joined ?
30          #    OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
31          ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
32          ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
33          ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
34          ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
35          # 3. a bare/relative name joins under a non-empty input_root
36          ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
37          ("sub/x.mp4", "local", "gs://bucket/videos/", "gs://bucket/videos/sub/x.mp4"),  #
root trailing slash
38          # 4. input_backend prefixes a bare bucket/key when no input_root
39          ("bucket/x.mp4", "gcs", "", "gcs://bucket/x.mp4"),
40          ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
41          # 5. bare/relative local path ? today's behaviour, unchanged
42          ("x.mp4", "local", "", "x.mp4"),
43          ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
44          ("C:\\v\\x.mp4", "local", "", "C:\\v\\x.mp4"),  # Windows path, default config:
unchanged (OS-independent)
45      ])
46      def test_resolve_input_uri(raw, ib, ir, expected):
47          assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
48
49
50      def test_resolve_input_uri_default_is_identity():
51          for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
52              assert resolve_input_uri(p) == p
53
54
55      # ------------------------------------------------------------- input_is_local
56      def test_input_is_local_true_for_local_paths():
57          assert input_is_local("/data/x.mp4") is True
58          assert input_is_local("rel/x.mp4") is True
59          assert input_is_local("local://x.mp4") is True
60
61
62      def test_input_is_local_false_for_buckets():
63          assert input_is_local("gs://b/x.mp4") is False
64          assert input_is_local("s3://b/x.mp4") is False
65          assert input_is_local("gdrive://Sharing/x.mp4") is False
66
67
68      def test_input_is_local_follows_input_root():
69          cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
70          assert input_is_local("x.mp4", cfg) is False           # bare name resolves into the
bucket
71          assert input_is_local("/abs/x.mp4", cfg) is True        # absolute path stays local
72          assert input_is_local("gs://other/x.mp4", cfg) is False
73
74
75      def test_input_is_local_accepts_pydantic_storage_config():
76          from backend.config.models import StorageConfig
77          sc = StorageConfig(input_root="gs://bucket/videos")
78          assert input_is_local("x.mp4", sc) is False
79          assert input_is_local("/abs/x.mp4", sc) is True
80
81
```

```
82      # ------------------------------------------------------------- acquire_local_input
83      def test_acquire_local_input_local_identity(tmp_path):
84          p = tmp_path / "v.mp4"
85          p.write_bytes(b"abc")
86          # Identity passthrough ? returns the SAME string, no copy, no temp dir.
87          assert acquire_local_input(str(p)) == str(p)
88
89
90      def test_acquire_local_input_relative_local_identity():
91          assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
92
93
94      def _capturing_fetch(captured):
95          def _f(uri, dst=None, *, config=None, overwrite=False):
96              captured["uri"] = uri
97              captured["dst"] = dst
98              captured["config"] = config
99              if dst:
100                 with open(dst, "wb") as fh:
101                     fh.write(b"remote-bytes")
102             return dst or uri
103         return _f
104
105
106     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
107         captured = {}
108         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
109         out = acquire_local_input("gs://bucket/videos/x.mp4", {"backend": "local", "gcs":
{}})
110         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # exact URI routed to
fetch
111         assert os.path.basename(out) == "x.mp4"                       # named from the ref
112         assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
113         # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
114         assert captured["config"] == {"backend": "local", "gcs": {}}
115
116
117     def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
118         captured = {}
119         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
120         acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
121         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # resolved BEFORE
fetch
122
123
124     def test_acquire_local_input_never_fetches_for_local(monkeypatch):
125         def _boom(*a, **k):
126             raise AssertionError("fetch must not be called for a local input")
127         monkeypatch.setattr(storage, "fetch", _boom)
128         assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
129         assert acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
== "clip.mp4"
130
```

11. assistant (2026-06-21T07:57:14.285Z)

The new test file `tests/test_input_source.py` is untracked (`??` in git status), not staged. This is a critical finding for the review ? but let me note this could be a side effect of the review snapshot. Let me verify whether it would be picked up by the test runner, and examine the key concern areas. Let me look at the reference patterns and path-safety helpers.

12. user (2026-06-21T07:57:15.236Z)

```
80      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
```

```
None:
81          with open(path, "w") as f:
82              json.dump({
83                  "video_id": video_id,
84                  "status": status,
85                  "segment_count": len(segments),
86                  "segments": [{"start": float(s.get("start_time", 0.0)),
87                                "end": float(s.get("end_time", 0.0))} for s in segments],
88              }, f, indent=2)
89
90
91      def cmd_infer(args: argparse.Namespace) -> int:
92          config = load_config(args.config) if args.config else load_config()
93          _apply_overrides(config, args.set)
94          storage_cfg = config.storage.model_dump()
95
96          scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97          os.makedirs(scratch, exist_ok=True)
98          config.output.output_dir = scratch
99
100         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102         print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105         video_id = compute_video_id(local_video)
106
107         # 3. VALIDATE (same registry as the main CLI).
108         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109         failures = [r for r in val_results if not r.passed]
110         db = Database(os.path.join(scratch, config.output.db_name))
111         db.add_video(video_id, local_video, "failed" if failures else "processed",
112                      [r.model_dump() for r in val_results])
113
114         segments: List[dict] = []
115         segmenter_actual = None
116         segmentation_ran = False
117         status = "failed"
118         try:
119             if failures:
```

13. user (2026-06-21T07:57:15.636Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Optional
4
5
6       def safe_output_filename(name: str) -> str:
7           """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9           `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12          """
13          if not name or not name.strip():
14              raise ValueError("output_filename must not be empty")
15          if "\x00" in name:
```

```
16              raise ValueError("output_filename must not contain a null byte")
17          if "/" in name or "\\" in name:
18              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21          return name
22
23
24      def resolve_under_root(path: str, root: str) -> str:
25          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26          rp = os.path.realpath(path)
27          rr = os.path.realpath(root)
28          try:
29              contained = os.path.commonpath([rp, rr]) == rr
30          except ValueError:  # different drives on Windows
31              contained = False
32          if not contained:
33              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34          return rp
35
36
37      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41          so the local 'point it at any of my videos' workflow is preserved. Setting an
42          allowed_root (a future hosted/tenant deployment) enforces containment.
43          """
44          if "\x00" in path:
45              raise ValueError("video_path must not contain a null byte")
46          if allowed_root:
47              resolve_under_root(path, allowed_root)
48
```

14. assistant (2026-06-21T07:57:18.572Z)

Now let me examine the critical hosted-mode (allowed_video_root) path. The LENS asks specifically: is hosted-mode rejection of a non-local URI tested? Let me look at the `/api/process` flow and how `allowed_root` is wired, plus the test `env` fixture.

15. user (2026-06-21T07:57:20.462Z)

```
255              )
256              if paths and isinstance(response, dict):
257                  response["reports"] = paths
258
259
260      @app.post("/api/process", status_code=202)
261      def process_video(req: ProcessRequest):
262          """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264          Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265          client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266          runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267          worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268          """
269          allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
270          try:
271              validate_input_path(req.video_path, allowed_root=allowed_root)
272          except ValueError as e:
273              raise HTTPException(status_code=400, detail=str(e)) from e
274          # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
```

```
URI can't be
275         # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it
(a fetch miss
276         # fails the job loudly). validate_input_path above still runs for every input: its
null-byte guard
277         # always applies, and a configured allowed_video_root (hosted mode) rejects a non-
local URI.
278         if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
279             if not os.path.exists(req.video_path):
280                 raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
281         if req.segmenter and not segmenter_registry.get(req.segmenter):
282             available = list(segmenter_registry.list_available().keys())
283             raise HTTPException(
284                 status_code=400,
285                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
286             )
287
288         job_id = uuid.uuid4().hex
289         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
290                                       req.player_pool, req.max_frames):
291             raise HTTPException(status_code=500, detail="Failed to persist job record.")
292         _get_executor().submit(_drain_due_jobs)
293         return JSONResponse(
294             status_code=202,
295             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
296         )
297
298
299     @app.get("/api/jobs/{job_id}")
```

16. user (2026-06-21T07:57:21.945Z)

```
3:import os
4:import subprocess
5:from unittest.mock import patch, MagicMock
7:import pytest
10:from fastapi.testclient import TestClient
12:import backend.main as m
13:from backend.infrastructure.database import Database
14:from backend.config.models import AppConfig
15:from backend.pipeline.validators.base import ValidationResult
26:@pytest.fixture
27:def env(tmp_path, monkeypatch):
37:def _video(tmp_path, name="m.mp4"):
43:def _pass():
```

17. assistant (2026-06-21T07:57:25.749Z)

Now I have the key finding forming: hosted-mode (allowed_video_root set) rejection of a non-local URI. Let me trace what `validate_input_path` does with a `gs://` URI when `allowed_root` is set. The claim in the comment (main.py:277) is that "a configured allowed_video_root (hosted mode) rejects a non-local URI". Let me verify this is actually true by tracing `resolve_under_root` with a `gs://` URI.

18. user (2026-06-21T07:57:27.169Z)

```
26      @pytest.fixture
27      def env(tmp_path, monkeypatch):
28          db = Database(str(tmp_path / "t.db"))
29          cfg = AppConfig()
30          monkeypatch.setattr(m, "db_instance", db)
31          monkeypatch.setattr(m, "global_config", cfg)
```

```
32          monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
33          client = TestClient(m.app)
34          return client, db, tmp_path, monkeypatch
35
36
37      def _video(tmp_path, name="m.mp4"):
38          p = tmp_path / name
39          p.write_bytes(b"not-a-real-video-but-hashable")
40          return p
41
42
43      def _pass():
44          return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
45
```

19. user (2026-06-21T07:57:30.404Z)

```
validate_input_path(gs://...) -> ValueError: path 'gs://bucket/x.mp4' escapes the allowed root
'/srv/videos'
realpath(gs://bucket/x.mp4) = C:\Users\avidu\Projects\badminton-highlight-
indexer\gs:\bucket\x.mp4
commonpath test (cwd-relative resolution shown above)
```

20. assistant (2026-06-21T07:57:34.899Z)

The hosted-mode rejection claim in the comment IS true (a `gs://` URI is treated as a relative path by `realpath`, which resolves it under cwd, which won't be under `/srv/videos`, so it raises). But it's rejected for an incidental reason (path-escape), not because it's a non-local URI ? and there's NO test asserting this behaviour. This is exactly the LENS's named gap. Let me verify the full test runs (including the untracked file) and confirm the CLI block has no test coverage.

21. user (2026-06-21T07:57:37.993Z)

```
tests/test_ablation.py:501:    monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d,
"--granularity", "group",
tests/test_ablation.py:517:    monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d,
"--litmus"])
tests/test_ablation.py:528:    monkeypatch.setattr("sys.argv", ["ablation", "--features-dir",
d])
tests/test_api_endpoints.py:55:    def process_video(self, video_id, video_path, *a, **k):
tests/test_api_endpoints.py:73:    r = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:88:    r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:99:    r = client.post("/api/process", json={"video_path":
"a\x00b.mp4"})
tests/test_api_endpoints.py:130:    r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:162:    r = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:174:    r2 = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:323:    rj = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:326:    rd = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_api_endpoints.py:333:    rf = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_api_endpoints.py:341:    """Records the video_path the segmenter actually received
(M2: must be a LOCAL path)."""
tests/test_api_endpoints.py:347:    def process_video(self, video_id, video_path, *a, **k):
tests/test_api_endpoints.py:348:        _CaptureSeg.last_path = video_path
tests/test_api_endpoints.py:361:    r = client.post("/api/process", json={"video_path":
```

```
str(vid), "segmenter": "x"})
tests/test_api_endpoints.py:370:    r = client.post("/api/process", json={"video_path":
"/no/such/local/file.mp4"})
tests/test_api_endpoints.py:388:    r = client.post("/api/process", json={"video_path":
"gs://bucket/videos/x.mp4", "segmenter": "x"})
tests/test_async_jobs.py:63:    def process_video(self, video_id, video_path, *a, **k):
tests/test_async_jobs.py:72:    def process_video(self, video_id, video_path, *a, **k):
tests/test_async_jobs.py:85:    assert {"id", "video_path", "video_id", "status", "progress",
"error", "result"} <= cols
tests/test_async_jobs.py:132:    req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")
tests/test_async_jobs.py:141:    rj = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
tests/test_async_jobs.py:160:    req_f = m.ProcessRequest(video_path=str(vid_file))
tests/test_async_jobs.py:162:    rf = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_async_jobs.py:170:        def process_video(self, video_id, video_path, *a, **k):
tests/test_async_jobs.py:174:    req_e = m.ProcessRequest(video_path=str(vid_file))
tests/test_async_jobs.py:176:    re = client.post("/api/process", json={"video_path":
str(vid_file)})
tests/test_async_jobs.py:190:    r = client.post("/api/process", json={"video_path":
str(vid_file), "segmenter": "x"})
```

22. user (2026-06-21T07:57:45.361Z)

```
tests/test_ablation.py
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_base.py
tests/test_config.py
tests/test_eval_harness.py
tests/test_eval_regression.py
tests/test_gemini_refine.py
tests/test_generate_highlights.py
tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py
tests/test_hit_detector.py
tests/test_job_worker_loop.py
tests/test_rally_seg_eval.py
tests/test_rally_slicer.py
tests/test_reingest_safety.py
tests/test_reporting_harvest.py
tests/test_storage.py
tests/test_upload_download.py
tests/test_watermark.py
tests/test_worker.py
tests/__pycache__/test_ablation.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_api_endpoints.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_async_jobs.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_base.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_config.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_config.cpython-313.pyc
tests/__pycache__/test_eval_harness.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_eval_harness.cpython-313.pyc
tests/__pycache__/test_eval_regression.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_eval_regression.cpython-313.pyc
tests/__pycache__/test_gemini_api.cpython-311-pytest-9.0.3.pyc
tests/__pycache__/test_gemini_api.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_gemini_refine.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_gemini_refine.cpython-313.pyc
tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_golden_fixtures.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_golden_real_fixtures.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_hit_detector.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_hit_detector.cpython-313.pyc
tests/__pycache__/test_job_worker_loop.cpython-313-pytest-9.0.3.pyc
```

```
tests/__pycache__/test_rally_slicer.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_rally_slicer.cpython-313.pyc
tests/__pycache__/test_reingest_safety.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_reporting_harvest.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_storage.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_upload_download.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_watermark.cpython-313-pytest-9.0.3.pyc
===check if any test invokes main() with argv===
tests/test_ablation.py:493:    """``main()`` runs the LOGO ablation and writes both a PDF and
the structured JSON."""
tests/test_ablation.py:503:    ablation.main()
tests/test_ablation.py:518:    ablation.main()
tests/test_ablation.py:530:        ablation.main()
tests/test_base.py:139:    # Exact duck-typing from main.py:156 and :213 and CLI :741
tests/test_config.py:233:    # The CLI override path: main() assigns args.output_dir onto the
typed model.
tests/test_config.py:323:    # positive that typed is used in pipeline code (global_config is
set at startup in main();
tests/test_golden_real_fixtures.py:480:    """red-team #6: --refresh-snapshot --all WITHOUT
--reason is a CLI error (argparse exits 2)."""
tests/test_golden_real_fixtures.py:483:    assert ei.value.code == 2  # argparse ap.error exit
code
tests/test_job_worker_module_seam.py:19:def test_symbols_are_attributes_of_backend_main():
```

23. assistant (2026-06-21T07:57:49.759Z)

Confirmed: there is NO test that invokes `backend.main.main()` (the CLI `--video-path` block). All the M2 tests exercise only the `/api/process` API path. The CLI block at main.py:588-637 has its own M2 logic (`input_is_local` gate + `acquire_local_input`) that is entirely untested. Let me confirm the CLI block is genuinely untested by checking the existing test files don't drive `main()` with `--video-path`, and run the actual test suite to verify the new tests pass and that the untracked file matters.

24. user (2026-06-21T07:58:04.547Z)

```
........................................                                 [100%]
42 passed in 10.96s
```

25. assistant (2026-06-21T07:58:08.846Z)

Tests pass. Now let me verify two more LENS concerns: (1) the `test_process_nonlocal_uri_skips_exists_and_fetches` test ? does it actually drive the worker through `_execute_processing`, and does it assert the *original URI* is kept in the DB record? (2) OS-dependent assertions. Let me check how the API job actually reaches `_execute_processing` and whether the DB record keeps the URI.

26. user (2026-06-21T07:58:13.363Z)

```
68:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
76:    _drain_due_jobs,
92:    video_path: str
140:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
157:    # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
159:    local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
165:    logger.info(f"Running validations for {req.video_path}...")
177:            # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
179:            db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
190:        db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
251:            db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
```

```
267:    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
271:        validate_input_path(req.video_path, allowed_root=allowed_root)
278:    if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
279:        if not os.path.exists(req.video_path):
280:            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
289:    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
292:    _get_executor().submit(_drain_due_jobs)
427:def run_cli_diagnose_clip(video_path: str, timestamp: float):
430:    print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
437:    results = registry.run_all(video_path, cfg)
448:    cap = cv2.VideoCapture(video_path)
531:        if not args.video_path:
547:        print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}...")
556:            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
563:        if not args.video_path:
566:        run_cli_diagnose_clip(args.video_path, args.debug_clip)
589:    if args.video_path and not args.server:
590:        print(f"[CLI] Processing video file: {args.video_path}")
594:        if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
```

27. user (2026-06-21T07:58:13.658Z)

```
43:    @property
44:    def name(self) -> str:
77:      3. A bare/relative name + a non-empty ``input_root`` ? joined under it
79:      4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
94:    if input_root:                    # 3. join a relative name under the configured root
===StorageRef def===
"""Storage value objects + URI parsing for the pluggable storage layer.

A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:

    <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4, /data/x.mp4)
    local://<path>              -> local   (local:///abs or local://rel/p)
    s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via endpoint_url)
    gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
    gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google Drive "My
Drive")

No cloud SDKs are imported here ? this module is pure stdlib so ``import backend.storage``
stays cheap.
"""
from __future__ import annotations

import re
from dataclasses import dataclass
from datetime import datetime
from typing import Optional

_SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
_KNOWN = ("local", "s3", "gcs", "gdrive")


@dataclass(frozen=True)
class StorageStat:
    """Lightweight object metadata (uniform across backends)."""
    size: int
    modified: Optional[datetime] = None
    etag: Optional[str] = None
    content_type: Optional[str] = None
```

```
@dataclass(frozen=True)
class StorageRef:
    """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and ``gdrive`` it
is
    empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
    backend: str
    bucket: str
    key: str
    uri: str

    @property
    def name(self) -> str:
        return self.key.rstrip("/").rsplit("/", 1)[-1]
```

28. assistant (2026-06-21T07:58:19.997Z)

Confirmed: the DB record keeps `req.video_path` (the original URI) at main.py:179, 190, 289 ? good. The `test_process_nonlocal_uri_skips_exists_and_fetches` test asserts the segmenter sees the local path but does NOT assert the DB record / video row stores the original `gs://` URI. That's a coverage gap for the "keep original URI for DB record + report" behaviour stated in the spec and the code comment.

Let me verify the test actually exercises `_execute_processing` (not just the submit gate) ? i.e. is `_InlineExecutor` defined and does the inline drain run the worker. And confirm the CLI's `input_is_local` short-circuit ordering is correct (so a non-existent local file still 404s).

29. user (2026-06-21T07:58:21.644Z)

```
tests/test_api_endpoints.py:18:class _InlineExecutor:
tests/test_api_endpoints.py:22:    def submit(self, fn, *args, **kwargs):
tests/test_api_endpoints.py:32:    monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
backend/main.py:68:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`,
`_get_executor`,
backend/main.py:76:    _drain_due_jobs,
backend/main.py:292:    _get_executor().submit(_drain_due_jobs)
backend/main.py:669:        _get_executor().submit(_drain_due_jobs)
```

30. user (2026-06-21T07:58:22.789Z)

```
16
17
18      class _InlineExecutor:
19          """submit() runs the fn synchronously ? async jobs complete before the response
20          is inspected (full lifecycle coverage lives in test_async_jobs.py)."""
21
22          def submit(self, fn, *args, **kwargs):
23              fn(*args, **kwargs)
24
25
```

31. assistant (2026-06-21T07:58:30.292Z)

The `_InlineExecutor` runs the drain synchronously, so `test_process_nonlocal_uri_skips_exists_and_fetches` genuinely exercises `_execute_processing` ? `acquire_local_input` ? `fetch` ? segmenter. Good ? the API integration-level identity/fetch path IS covered.

Now let me confirm the identity-no-copy guarantee at the integration level. The test `test_process_local_input_passes_identity_path` asserts `_CaptureSeg.last_path == str(vid)` ? that's the segmenter receiving the exact local path. Good, that IS asserted at integration level for the API. But the LENS specifically flags the CLI path. Let me verify the CLI block's

behaviour and confirm no test covers it, then look at whether the `validate_input_path` null-byte check still runs before `input_is_local` in the API (ordering).

Let me also double check one subtle thing: in the API, `input_is_local` is called with `req.video_path` which has already passed `validate_input_path`. But for a `gdrive://` URI in hosted mode ? does it get rejected? Let me verify the comment's claim holds for gdrive too and check a potential edge: a relative local path in hosted mode.

32. user (2026-06-21T07:58:33.363Z)

```
gdrive://Sharing/x.mp4                  -> rejected
gs://b/x.mp4                            -> rejected
s3://b/x.mp4                            -> rejected
local://srv/videos/x.mp4                -> rejected
```

33. user (2026-06-21T07:58:33.750Z)

```
588         # CLI processing mode (no web UI needed)
589         if args.video_path and not args.server:
590             print(f"[CLI] Processing video file: {args.video_path}")
591             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
592             # The local-existence fast-fail only applies to a local input; a remote fetch
fails loudly.
593             storage_cfg = getattr(global_config, "storage", None)
594             if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
595                 print(f"[FAIL] Video file not found: {args.video_path}")
596                 return
597             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
598
599             video_id = compute_video_id(local_video)
600             was_reingest = db_instance.get_video(video_id) is not None
601
602             # Validate (with-context variant surfaces ffprobe_meta for the report)
603             print("[CLI] Validating...")
604             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
605             failures = [r for r in val_results if not r.passed]
606
607             # Save video status
608             status = "failed" if failures else "processed"
609             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
610
611             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
612             segmenter_actual = None
613             segmentation_ran = False
614             try:
615                 print("\n" + "="*40)
616                 print("VALIDATION SUMMARY")
617                 print("="*40)
618                 for r in val_results:
619                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
620                     print(f"{status_symbol} {r.validator_name}: {r.message}")
621                 print("="*40 + "\n")
622
623                 if failures:
624                     print("[FAIL] Video failed checks. Indexing halted.")
625                     return
626
627                 # Index
628                 segmenter_cls = segmenter_registry.get(seg_name)
629                 if not segmenter_cls:
```

```
630                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
631                     return
632
633                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
634                 segmenter_actual = seg_name
635                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
636                 segmenter = segmenter_cls(db_instance, global_config)
637                 result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
638                 segmentation_ran = True
639
```

34. assistant (2026-06-21T07:58:44.037Z)

The CLI block mirrors the API logic correctly and keeps `args.video_path` (original) in the DB record at main.py:609 while using `local_video` for processing. The hosted-mode rejection works for all schemes (incidentally, via path-escape).

Now I have a complete picture. Let me run a final verification: confirm the full suite still collects/passes including the untracked test, and double-check there are no OS-dependent test assertions that would break on CI. The test file author was careful (lines 9-11, 44 note OS-independence). Let me verify the one potentially-fragile assertion ? `os.path.basename(out) == "x.mp4"` in the remote-fetch test works cross-OS.

35. user (2026-06-21T07:58:48.225Z)

```
ref.name = 'x.mp4'
```

36. user (2026-06-21T07:58:59.242Z)

```
.................................................................... [ 80%]
..................                                                   [100%]
90 passed in 8.03s
```

37. assistant (2026-06-21T07:59:10.266Z)

All green and OS-independent. I have a complete assessment. Let me compile findings.

Summary of my analysis on test quality & coverage:

1. **`tests/test_input_source.py` is UNTRACKED** (`??` in git status), not staged. The spec lists it as a deliverable. If the PR is cut with `git add <explicit paths>` and this file is forgotten, the entire `resolve_input_uri` / `input_is_local` / `acquire_local_input` unit suite (the bulk of M2 coverage) silently never ships. This is the highest-impact finding.

2. **CLI `--video-path` block (main.py:588-637) has ZERO test coverage.** All 3 new API tests plus the unit tests exercise `resolve_input_uri`/`storage` helpers and the `/api/process` path ? but no test invokes `backend.main.main()` with `--video-path`. The CLI block has its own copy of the M2 logic (the `input_is_local(...) and not os.path.exists(...)` gate at :594, `acquire_local_input` at :597, and original-URI-in-DB at :609). A regression in that block ? e.g. dropping the `input_is_local` guard so a `gs://` URI gets a bogus "file not found", or passing `args.video_path` instead of `local_video` to the segmenter ? would not fail any test.

3. **Hosted-mode (`allowed_video_root` set) rejection of a non-local URI is asserted nowhere.** The code comments at main.py:277 explicitly claim "a configured allowed_video_root (hosted mode) rejects a non-local URI." I verified this holds ? but only incidentally (the URI fails the path-escape check in `resolve_under_root`, not because it's recognized as non-local). No test pins this behaviour, so it's fragile.

4. **The "keep original URI for the DB record" guarantee is not asserted.** The non-local-URI test asserts the segmenter sees the fetched local path, but never checks that the persisted video/job row stores the original `gs://?` URI (main.py:179/190/289/609).

Let me record these.

38. user (2026-06-21T07:59:42.327Z)

Structured output provided successfully

39. assistant (2026-06-21T07:59:53.600Z)

I've completed the M2 review through the test-quality & coverage lens. Summary of findings:

Implementation is correct ? I verified `resolve_input_uri` precedence, the identity-no-copy passthrough, OS-independent anchored-path detection (deliberately avoiding `os.path.isabs`, which differs on Windows+py3.13), cross-OS-safe `basename` assertions, and that both the API and CLI call-sites keep the original URI in the DB while processing the resolved local path. The 42 M2 tests pass; the helper layer and test assertions are OS-independent (the author explicitly handled the cross-OS CI concern at `tests/test_input_source.py:9-11,44`).

Four findings, all about test coverage rather than logic bugs:

1. **BLOCKER** ? `tests/test_input_source.py` is untracked (`??`), absent from `git diff origin/master`. Under this repo's explicit-path staging rule, it risks being left out of the commit, shipping M2 with its entire unit suite missing. File: `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_input_source.py`.

2. **MAJOR** ? The CLI `--video-path` block (`backend/main.py:588-637`) has zero coverage; no test invokes `backend.main.main()`. Its independent copy of the M2 gate/acquire/persist logic could regress invisibly.

3. **MAJOR** ? Hosted-mode (`allowed_video_root` set) rejection of a non-local URI ? an explicit safety claim in the code comment at `backend/main.py:277` ? is asserted nowhere, and holds only incidentally via `realpath`/`commonpath` path-escape. Every M2 test runs with `allowed_video_root` unset.

4. **MINOR** ? The non-local-URI test asserts the segmenter sees a local path but never asserts the DB record keeps the original `gs://` URI (the other half of the M2 contract at `backend/main.py:179,190,289`).

Findings recorded via StructuredOutput with exact assertions for each missing test.